

Golden Farm — Registro de cambios

Lunes 17 y martes 18 de agosto de 2026

Para diseño

Este documento recoge **solo los cambios de esta tanda**. Cada apartado dice qué se cambió, por qué, y qué se comprobó antes de subirlo. El registro completo del proyecto son 109 páginas y está en CHANGELOG.md.

Día 22 — Lunes 17/08 • Se cae el panel de balanceo y se arregla el arranque

Fuera todo el HTML que no es el juego

Decisión de dirección: **el único HTML que queda es public/index.html**, el que abre el juego. Se borraron public/balance.html, public/vista-ventanas.html y public/game/balance.js.

El motivo no es orden, es que ese panel hacía daño: guardaba valores en una tabla de la nube que **pisaban al código al arrancar**, y eso nos costó tres problemas seguidos que parecían bugs del juego y no lo eran (timers fantasma, la respuesta al clic que “volvía sola” y, ahora, un arranque trabado). A partir de acá **manda el código**: los números viven en state.js y config.js, y lo que se ve es lo que está escrito.

También se sacó await window.BAL_READY de main.js y los comentarios que todavía mandaban a editar cosas en balance.html.

Por qué la barra de carga se plantaba a la mitad

Dos causas encadenadas, las dos reales:

1. **index.html seguía pidiendo game/balance.js**, que ya no existía. El cargador reintenta cada script 8 veces con espera creciente antes de rendirse: eran ~10 segundos de nada en cada entrada.
2. **18 sprites no estaban en el atlas** y se bajaban sueltos: los tableros, el buzón, el baúl de premios, el montículo, la isla y — el hallazgo grueso — las etapas de crecimiento cropm_/cropg_ de ciruela, cereza y remolacha. El armador del atlas filtraba por el prefijo crop_, que **no** atrapa a cropm_ ni a cropg_: cada cultivo nuevo entraba con su ícono pero sus dos etapas quedaban afuera. Esos 18 pedidos caían en la fase de reintentos de boot.js, y ahí la barra se dibuja justo a la mitad. No era el bosque ni el peso: era esperar.

Arreglado: cropm_/cropg_ agregados a los prefijos del armador, los one-off agregados a mano, atlas reconstruido (604 sprites, 790 KB) y ?v=46 para romper caché. Verificado por script: **assetList pide 414 sprites y los 414 están en el atlas — cero pedidos sueltos.**

Vigía de arranque (para no depender más de la consola)

Si algo falla mientras está la pantalla de carga, ahora **el error se escribe en la propia pantalla**, en un recuadro legible. Y si a los 20 segundos seguimos ahí sin ningún error, el vigía cuenta en qué paso quedó y a qué altura está la barra. Alcanza con una foto de la pantalla: se terminó el F12.

El error real del arranque: un bloque de código pegado en el lugar equivocado

Con el vigía puesto, el juego lo dijo solo en pantalla: W is not defined → FarmScene is not defined.

Un edit anterior había pegado las 18 líneas de **la forma del claro del bosque al principio de farm.js**, fuera de toda función. Ahí W (el ancho del mundo) no existe, así que el archivo reventaba en la línea 7 y **nunca llegaba a declarar FarmScene**. De paso, dentro de dibujarBosque() había quedado la versión vieja, que llamaba a met() sin tenerlo definido: o sea que el bloque no solo estaba de más arriba, estaba **faltando abajo**. Movido a su lugar.

Queda un aprendizaje de método: node --check no lo agarraba porque **la sintaxis era válida** — el error era de ejecución. Ahora la verificación corre cada archivo entero y comprueba que declare su escena (FarmScene, BootScene, ForestScene, PlazaScene). Eso sí lo habría cazado.

La forma del bosque queda rectangular

Dirección: *“la forma cuadrada creo que queda mejor que el resto, esta forma está bien”*. BOSQUE_REDONDEZ y BOSQUE_ONDA a **0**. El claro rectangular acompaña a la grilla de celdas, que también es rectangular, y por eso lee mejor que el óvalo. La irregularidad del borde la sigue dando BOSQUE_JITTER (el desorden de cada árbol), que alcanza para que no parezca dibujado con regla. Los parámetros quedan en el código por si algún día se quiere volver a probar: es cambiar un número.

La granja se comprime: el mundo pasa de 23x15 a 17x12

Dirección, viendo la granja alejada: *“todo el corral, todos los edificios y todos los nodos deben estar más comprimidos”*. Los números le daban la razón sin discusión: **345 celdas para 73 de contenido (21%)**, con las filas 0, 1, 2, 6 y 14 **enteras vacías** y los 18 nodos exiliados en las columnas 16 a 22, lejísimos del granero.

Mundo nuevo: **17x12 = 204 celdas, 36% ocupado, sin una sola fila muerta**. Todo el contenido vive entre las columnas 1-15 y las filas 2-10:

- **Izquierda:** las 12 parcelas arriba (1-4 / 2-4), el Establo en el medio, la laguna abajo.
- **Centro:** el granero arriba con el tablón a su izquierda y el buzón + baúl a su derecha (como pidió dirección), y debajo la columna de oficios en bandas de dos filas — cocina y horno, mercado y altar, herrería y curtiduría.
- **Derecha:** los 6 árboles en tres pares, y debajo las 6 piedras y los 6 minerales.

Las posiciones se reescribieron **sin tocar el orden de WORLD_OBJECTS**: los layouts guardados de los jugadores referencian los objetos por índice, así que reordenar la lista les movería la granja.

El bosque ya no se sube a la cerca

Dirección: *“el corral pasa por encima de los árboles”*. Cierto, y la primera corrección fue **equivocada**: subí BOSQUE_COLCHON, que no es la palanca — ese número solo agranda el radio de referencia. El hueco real lo da BOSQUE_AIRE, porque el borde del claro cae en mitad del mundo + AIRE x radio.

Medido: con AIRE = 0,05 el árbol de la derecha se metía **32 px dentro** del mundo y el tronco de la fila de arriba colgaba **15 px por debajo** del borde, justo encima de la cerca. Ahora 0,23, que es la cuenta para dejar una celda entera de césped teniendo en cuenta el ancho del sprite (105 px) y que el punto de medición está al 72% de su alto — de ahí venía el error.

tools/preview-granja.py — aprobar el layout sin deployar

Cada vez que se movía un edificio había que deployar y entrar a mirar. Ahora este script lee las **mismas** posiciones que usa el juego (ejecuta config.js de verdad con node, no las reescribe) y arma un PNG con los sprites reales, el césped, la cerca y el anillo de bosque. Fue lo que dejó al descubierto lo del AIRE: en la primera pasada se veía el bosque pisando la cerca igual.

El borde blanco era la orilla del mar

Dirección, sobre una captura: *“ese borde blanco quedó la orilla de mar”*. Exacto. drawOlas() pintaba dos rectángulos redondeados blanquecinos —la espuma de la costa— y su objeto gráfico se creaba dentro de un if (GF.ISLA) que seguía en true. O sea: cambiamos el mar por bosque, pero **la espuma de la orilla sobrevivió al cambio** y quedó flotando sobre el césped como un contorno fantasma en las esquinas.

Al abrir ese bloque apareció algo peor: **el bosque se dibujaba dentro de if (this.textures.exists("isla"))**. El anillo entero colgaba de que cargara la textura del mar, que ya no se usa para nada. Si ese PNG faltaba, no había bosque.

Los dos caminos quedaron separados: if (GF.BOSQUE) → bosque / else if (GF.ISLA) → mar, costa y olas. GF.ISLA se deja en true a propósito: es el respaldo del interruptor? sinbosque=1, que sin esto abriría el juego sobre un vacío liso. De paso, las nubes ahora cruzan el margen del bosque (420) y no el de la isla (260), que las cortaba a media pantalla.

El buzón tapaba el baúl

Al comprimir el mundo, el buzón quedó en la fila 3 y el baúl en la 2. En un juego con profundidad por fila eso no es “un poco más abajo”: es **delante**, así que el buzón se dibujaba encima del baúl y lo tapaba. Los tres vuelven a la misma fila que el granero, en el orden de siempre: granero (columnas 6-8), buzón (9), baúl (10).

Lección para el resto del layout: cuando se mueve algo, la fila no es solo posición vertical, también es orden de dibujado. Dos objetos que se quieren “al lado” tienen que compartir fila.

El bosque se ordenaba mal: por el techo del árbol en vez de por su base

Dirección: *“hay árboles que deben estar por detrás de los que están más cerca del corral, por proximidad”*. Tenía razón, y el error era de una línea.

dibujarBosque() ordenaba con `lista.sort((a,b) => a[0] - b[0])`, donde `a[0]` es **py: el borde de ARRIBA del sprite**. Pero cada árbol se dibuja con su propia escala (0,92 a 1,26), así que dos árboles que arrancan a la misma altura **apoyan hasta 37 px distinto**. Como la fila del bosque mide 18 px, el error valía hasta **2,1 filas**: un árbol grande del fondo se dibujaba antes que uno chico de adelante y su tronco asomaba por delante del que estaba más cerca del corral.

Ahora ordena por `py + alto x escala`, o sea por **dónde apoya**, que es el mismo criterio con el que el resto del juego ordena todo (`baseRow`, no el `techo del sprite`). De los 3.349 árboles del anillo, prácticamente todos cambian de posición en la lista.

La vista previa (`tools/preview-granja.py`) replica el algoritmo del bosque, así que se corrigió igual: si no, mostraría un bosque que el juego ya no dibuja.

El bosque, medido en vez de discutido

Dirección: *“no hace falta poner tres árboles detrás de uno; con dos a los costados del que está más adelantado ya cubrés”* y *“ten en cuenta que un árbol ocupe solo una celda”*.

Se hicieron dos herramientas para dejar de opinar a ojo: `tools/medir-bosque.py` (cuenta árboles y % de fondo visible) y `tools/comparar-bosque.py` (renderiza el borde con distintas configuraciones). Lo que salió:

- **Agujeros de fondo NO había.** Ni con un 80% menos de árboles. El anillo estaba masivamente sobredibujado: 3.349 árboles, la enorme mayoría tapados por completo.
- **Lo que molestaba era otra cosa:** por las rendijas entre troncos asomaba **la fila de atrás**. Medido en el sprite, el tronco tiene 23-27 px y las columnas van cada 25 px: deberían tocarse. El culpable era el **jitter horizontal de ±8 px**, que podía separar dos troncos hasta 41 px.
- **Incoherencia de escala encontrada de paso:** los árboles del BOSQUE medían 2,3 a 3,2 celdas de ancho, o sea **más grandes que los que se talan en la granja**, que miden 2. El fondo era más grande que el primer plano.
- **“Un árbol = una celda” se probó y no funciona.** La copa del sprite es ancha; a 42 px deja de leerse como árbol y hacen falta 300 para cerrar la masa, y aun así queda 1,8% de fondo a la vista. Separándolos salta a 19% y 37%. A **2 celdas** —el mismo tamaño que los de la granja— alcanzan 126 con 0,02%.

Cambios: `BOSQUE_TAM` (ancho en `CELDAS`, así no se puede volver a desincronizar del resto del juego), `BOSQUE_TRABA` (filas alternas corridas media columna: el de atrás cae en el hueco de los dos de adelante, que es la idea de dirección aplicada), y

BOSQUE_JITTER_X / _Y separados. BOSQUE_PASO a 0,65, elegido probando 0,90 / 0,75 / 0,65 / 0,55 sobre el borde real: a 0,90 queda dentado y asoma el fondo, a 0,55 vuelve a ser una pared.

El pasto ahora llega hasta el bosque

La franja entre la cerca y el primer árbol —que existe a propósito— estaba pintada con el verde liso del fondo de la cámara, porque el mosaico de pasto cubría solo el mundo jugable. Se veía como un anillo plano alrededor de la granja. Ahora el `renderTexture` del pasto se extiende hasta el margen del bosque, en lote. Los tiles de más quedan tapados por los árboles y no cuestan nada.

[tools/editor-bosque.html](#) — el banco de pruebas del bosque

Dirección: “¿puedes generarme un HTML donde yo poner los árboles y mostrarte cómo debería quedar?”.

Hecho, y con una condición que viene de la lección del panel de balanceo: **vive en tools/, no en public/**. No se deploja, no habla con la nube y no puede escribir un solo valor del juego. Solo lee y devuelve texto. La regla que quedó es clara: las herramientas que ESCRIBEN en el juego terminan rompiéndolo; esta exporta números que se pasan a mano.

Es un archivo suelto que se abre con doble clic, sin servidor: el arte (árbol, césped y cerca) va incrustado en base64, así que funciona desde cualquier carpeta. Y como va incrustado y no como ruta externa, el lienzo no queda bloqueado y **puede medirse en vivo el % de fondo a la vista** mientras se colocan los árboles.

Segunda versión, por dirección: “dibuja la cuadrícula y centra el árbol en cada celda pegado al borde de abajo”. Se abandonó la colocación a mano libre: ahora hay **un árbol por CELDA**, centrado horizontalmente y apoyado en la línea de abajo de su celda, con la cuadrícula dibujada POR ENCIMA de los árboles para no perderla de vista. Es como razona el resto del juego —todo se ancla a la grilla por su base—, así que lo que se compone ahí es trasladable tal cual, sin traducir coordenadas sueltas a nada.

Qué hace: clic planta en la celda, arrastrar planta varias de una (pincel), clic derecho borra —también arrastrando—, Ctrl+Z deshace, la rueda cambia el tamaño. Dibuja ordenando por FILA, igual que el juego. Botones de *Llenar todo* y *Damero* para partir de algo en vez de un lienzo vacío. El de copiar devuelve la lista de celdas, la ocupación y cuántos árboles hay por fila.

Las tres leyes de colocación del bosque

Dirección, 17/8: además de la celda, “que los árboles se puedan poner en la encrucijada entre cuatro celdas”. Y una aclaración importante que queda escrita en la propia herramienta: **estas reglas son SOLO para dibujar el bosque. Dentro de la granja no rigen** — ahí cada objeto se sigue anclando como siempre (`leftCol + baseRow`, un objeto por celda, sin encrucijadas).

LEY 1 · CELDA centrado en la celda, apoyado en su borde de abajo
 $x = (gx + 0,5) \times 42$ base = $(gy + 1) \times 42$

LEY 2 · ENCRUCIJADA en el cruce de cuatro celdas, o sea en un vértice de la cuadrícula
 $x = gx \times 42$ base = $gy \times 42$

LEY 3 · MEDIA ARISTA a la mitad exacta de una arista VERTICAL de la celda:
sobre la línea,
pero a media altura
 $x = gx \times 42$ base = $(gy + 0,5) \times 42$

El patrón es limpio: ley 1 = *x a media celda + y entera* · ley 2 = *x entera + y entera* · ley 3 = *x entera + y a media celda*. Falta la cuarta combinación —*x a media + y a media*, que sería el centro de la celda—: no está pedida, pero el sitio existe y se puede sumar cuando se quiera. Con las tres, los anclajes caen en una rejilla de **media celda (21 px)**, que es la resolución más fina posible sin salirse de la cuadrícula del juego.

Lo interesante es que las rejillas quedan **a media celda unas de otras**. Usadas juntas dan exactamente el trabado que veníamos persiguiendo a mano con BOSQUE_TRABA y el jitter —cada árbol de una rejilla cae en el hueco de la otra— pero expresado como una regla que se puede dibujar y razonar, en vez de un número suelto que hay que adivinar. Si esta composición gusta, el algoritmo del juego se puede reescribir directamente sobre las dos leyes y desaparecen TRABA, PASO y FILAS como números mágicos.

El editor tiene las tres leyes como interruptores independientes (se pueden activar y desactivar por separado, siempre con una encendida como mínimo), dibuja las anclas — punto verde para la celda, aspa azul para la encrucijada, barrita rosa para la media arista— y el clic salta a la más cercana entre las activas.

La composición a mano resultó ser una regla — y el bosque se reescribe sobre ella

Dirección compuso el borde a mano en tools/editor-bosque.html y exportó 675 árboles. Al contar, el patrón salió solo:

ley

celda

encrucijada

media arista

O sea: puso **las tres leyes en todos sus anclajes** y después **abrió claros a mano** en dos de ellas. Eso —y no la separación— es lo que impide que el bosque se lea como una retícula.

dibujarBosque() deja de generar con separaciones y pasa a recorrer los anclajes de las tres leyes, con una densidad por ley (GF.BOSQUE_DENSIDAD) que reproduce ese raleo. Desaparecen como sistema BOSQUE_PASO, BOSQUE_FILAS y BOSQUE_TRABA: cinco números que nadie sabía justificar, sustituidos por tres reglas geométricas y dos porcentajes medidos.

Nota de método, para no repetirlo. Antes de tener el export intenté deducir la composición midiendo píxeles de las capturas. Fue tiempo perdido y, peor, dio tres respuestas distintas para la misma imagen según dónde recortara: 68%, 41% y 4% de cobertura de tronco. La conclusión no era “medir mal”, era que **la medición no servía para esa pregunta**. El export de un clic dio la respuesta exacta en un segundo. Cuando existe el dato, no se estima.

Por qué el bosque del juego no se veía como la composición a mano

Dirección: “¿por qué no se ve como el que hice?”. La respuesta estaba en su propio export, en dos líneas fáciles de pasar por alto:

"tamMin": 2,
"tamMax": 2

Todos sus árboles miden exactamente lo mismo. El juego los variaba $\pm 15\%$ (BOSQUE_ESC_VAR). Con variación, dos árboles del mismo anclaje apoyan a alturas distintas: la franja de troncos se ensancha y se emborrona, y se pierde la línea limpia. Con todos iguales, los troncos de cada fila quedan alineados al pixel.

BOSQUE_ESC_VAR pasa a **0**. La variedad que hace falta ya la da el volteo horizontal aleatorio y —sobre todo— los claros del raleo. La variación de tamaño no aportaba riqueza, aportaba ruido.

Vale la pena anotar el patrón: las tres últimas diferencias entre lo compuesto y lo generado —orden por la base, tamaño de 2 celdas, tamaño uniforme— **estaban todas en los datos**, y ninguna se resolvió mirando la imagen. Se resolvieron leyendo el export.

El raleo va hacia adentro, no en el frente

Dirección: “*sigue sin verse igual*”. Al volver al export y leerlo **banda por banda** —en vez de como un total— apareció lo que faltaba:

- En las dos bandas pegadas al claro puso **todos** los árboles: 26 de 26 y 27 de 27. Los claros los abrió **atrás**. Mi raleo era parejo y agujereaba justo la primera línea, que es la única que se ve entera.
- Y su primera banda tiene **solo árboles de la ley de CELDA**, uno cada 42 px. La ley de encrucijada comparte base con la de celda, así que juntas dejan un árbol cada 21 px y la línea cierra del todo, como un muro corrido.

Dos reglas nuevas, las dos leídas del export: BOSQUE_FRENTE_SOLIDO (celdas de bosque sin ralear junto al claro) y, dentro de esa franja, la ley de encrucijada se calla.

Aviso sobre la métrica de “proporción de madera”. Se usó para comparar y ha fallado tres veces, dando 68%, 41%, 4% y 100% sobre imágenes equivalentes, según qué fila tomara: dentro del bosque cualquier fila da 100%, y la fila del frente depende de dónde se recorte. **No es de fiar y no debe usarse para decidir.** Lo que vale es mirar el render.

La regla, dicha en una frase por dirección

Después de varias rondas deduciendo el patrón del export, dirección lo dijo directamente:

“una fila de árboles cubriendo el centro inferior de cada celda, la siguiente fila ocupando la mitad de las líneas en vertical, y repetir con los dos siguientes, y así”

Se **alternan dos leyes**, y la de encrucijada no entra:

banda 1	CELDA	$x = (col + 0,5) \times 42$	$base = (fila + 1) \times 42$
banda 2	MEDIA ARISTA	$x = col \times 42$	$base = (fila + 0,5) \times 42$
banda 3	CELDA	...y así	

Las bandas caen cada 21 px alternando, y cada una queda corrida media celda respecto de la anterior: **ese es el entrelazado**, y sale solo de la geometría. La encrucijada sobraba porque comparte base con la celda: juntas dejaban un árbol cada 21 px y la línea cerraba como un muro.

GF.BOSQUE_LEYES = "cv" y filas enteras. Con esto el bosque queda descrito por **una frase**.

Lo que costó llegar, y por qué

Se dieron varias vueltas intentando deducir la regla midiendo píxeles y contando totales del export. Dos aprendizajes que conviene no repetir:

1. **La métrica de “proporción de madera” no servía.** Dio 68%, 41%, 4% y 100% sobre imágenes equivalentes según qué fila tomara. Cuando una medición da resultados incompatibles para la misma pregunta, el problema no es afinarla: es que no mide lo que se cree.
2. **Los totales del export escondían la estructura.** Contar 260/187/228 llevó a “las tres leyes con raleo”, que era una descripción estadísticamente parecida pero estructuralmente falsa. La estructura solo apareció al leerlo **banda por banda**, y quedó clara del todo cuando dirección la enunció. Ante un patrón, preguntar la intención ahorra más que medirla.

Los cuatro lados del bosque, visibles de una vez

Dirección: *“hice el máximo de zoom out y no puedo arrastrar la cámara hacia arriba; creo que los 4 lados del bosque deben ser visibles porque lo que interesa es que la granja se pueda expandir”.*

Medido, y el problema era de criterio, no un fallo suelto: el zoom mínimo estaba calculado para que **nunca se viera el borde del dibujo**, y el efecto secundario era que el anillo no entraba. Con una pantalla de 1341x630 la vista al alejar medía 1522x715 sobre un anillo de 1522x1312: **597 px de bosque fuera de cuadro**, arriba y abajo, alcanzables solo arrastrando.

Tres cambios:

- **El anillo pasa a tener margen por eje**, y es más ancho que alto: `BOSQUE_MARGEN_X = 1150` y `_Y = 460`. Total 3014x1424, relación 2,14:1 — la de una pantalla. Antes era casi cuadrado, que es justo la forma que no entra en un monitor.
- **El zoom mínimo pasa de “llenar” a “encajar”** (`max → min`): al alejar del todo entra el anillo entero, con la granja centrada y bosque a los cuatro lados.
- **El fondo de la cámara pasa a ser el verde medio de las copas** (0x2f5a28, muestreado del sprite): si en una pantalla muy panorámica sobra un poco, se lee como bosque lejano.

El fondo del bosque se rompe; el frente no se toca

Al ver el anillo entero por primera vez apareció algo que de cerca no se notaba: con todos los árboles idénticos y en retícula perfecta, **el interior se leía como papel pintado**.

La regla de dirección está pensada para el borde, que es lo que se ve de cerca. Así que el desorden y el raleo se aplican **solo pasado BOSQUE_FRENTE_SOLIDO**: la primera línea queda exactamente como la compuso, y el fondo se rompe con un 14% de claros y 6 px de desorden. Para que el azar no baile entre una zona y otra, los números aleatorios se sacan **siempre**, se usen o no.

El mapa pasa a ser cuadrado: 1600 x 1600, granja al centro

Dirección: *“el mapa debe ser igual para alto y ancho, y la granja en el centro, 1600x1600”*.

Los márgenes se calculan solos desde GF.MAPA, porque la granja (714x504) **no** es cuadrada y por tanto los márgenes tampoco pueden serlo: 443 px a los lados y 548 arriba y abajo. Lo que queda igual es el total. Si algún día cambia el tamaño del mundo, la cuenta se rehace sola.

Consecuencia medida, y cómo se resolvió. Un mapa cuadrado en una pantalla panorámica no encaja: para ver los 1600 de alto hay que alejar hasta que entren ~3.300 de ancho, así que sobran **1.770 px a los lados** — más ancho que el propio mapa. Antes ahí se veía color liso.

Ahora se rellena con un **mosaico del propio bosque**, generado con las mismas leyes: una textura de 336x336 (8x8 celdas) dibujada con envoltura —cada árbol se pinta también desplazado ± 336 , así que el que cruza un borde reaparece por el otro— y estirada en un `tileSprite` por debajo de todo. El resultado es que **el mapa cuadrado deja de tener un “afuera” visible**: se ve bosque que sigue. Coste: una textura de 336x336 generada una sola vez.

La vista previa aprende a dibujarlo también: `EXTRA=700 python3 tools/preview-granja.py` renderiza el mapa con 700 px de bosque alrededor, para comprobar que no asoma ningún borde.

El mosaico del fondo salía en bandas — dos fallos, los dos míos

Dirección, sobre el juego ya deployado: *“no se ve como lo que me mostrás”*. Ciertamente: la mitad derecha de la pantalla salía en **bandas horizontales de troncos repetidas**, con una costura vertical dura contra el anillo. Dos causas:

1. **Orden de profundidad en la envoltura.** Para que el mosaico no tenga costura, cada árbol se dibuja también desplazado ± 336 . Yo ordenaba por la base del ORIGINAL y desplazaba al dibujar, así que la copia que envolvía desde abajo se pintaba tarde arriba y su tronco tapaba a los que iban delante. De ahí las bandas. Ahora las copias entran en la lista **como árboles propios, con su base ya desplazada**, y se ordena después. (Es exactamente el mismo error de ordenar por el sitio equivocado que ya había aparecido con py vs base y con el buzón. Tercera vez.)
2. **El mosaico no estaba alineado con la retícula del mundo**, así que arrancaba donde cayera el borde del tileSprite. Se corrige con $\text{tilePositionX/Y} = \text{la esquina del sprite en coordenadas de mundo, módulo } 336$. Como el mosaico mide 8 celdas justas, cualquier múltiplo de 42 encaja.

Comprobado antes de deployar reproduciendo el mosaico del juego en Python y **tileándolo 3x3** para buscarle la costura: no la tiene.

Los dos fallos que impedían mover la cámara

Dirección, con el navegador al 25%: *“es como si todo no tenga posición relativa, ahí yo no puedo arrastrar la cámara hacia arriba”*. Dos fallos independientes, los dos míos, y los dos por asumir en vez de comprobar cómo funciona Phaser.

1. Mis límites de arrastre estaban mal. Yo recortaba a mano el scroll suponiendo que scrollX/scrollY es el borde visible de la cámara. **No lo es cuando el zoom no vale 1**: en Phaser el área visible se CENTRA y mide alto / zoom. Mi fórmula y la suya divergen con el zoom, y con el navegador al 25% los dos rangos **ni se solapaban**: mi código ponía un valor, el recorte de Phaser lo corregía a otro, y la cámara quedaba clavada. Fuera los recortes manuales (había cuatro, uno por cada sitio que panea): la cámara ya tiene `setBounds`, que recorta bien.

2. El tope de alejado del jugador era un número fijo. La rueda recortaba a $[0,6 \dots 2,2]$. Con el navegador al 25% el zoom base sube a $\sim 5,2$, así que el 0,6 dejaba el mínimo alcanzable en $\sim 3,1$ y el tope duro de 2,2 lo pisaba: el jugador **nunca** podía llegar al 1,61 que hace entrar el mapa. Ahora el rango sale de la cuenta ($\text{zoomUserMin} = \text{zMin} / \text{zBase}$), así que vale igual con el navegador al 25%, al 100% o al 300%. Verificado en los cuatro casos: el mapa entra en todos.

Lección, y van varias del mismo tipo: **cuando mi cuenta y la de la librería dan cosas distintas, la que sobra es la mía**. Phaser ya sabía recortar el scroll; yo dupliqué esa lógica mal.

El corte de la laguna y el del césped

Dirección: *“el corte en la laguna y decoración del césped”*. Dos cosas distintas, las dos reales.

La laguna estaba deformada. Se estiraba a la caja de celdas sin mirar su proporción: pond.png mide 107x93 (relación 1,15) y la caja 178x136 (1,31), o sea **un 14% de deformación horizontal**. La forma redonda salía aplastada, y eso es lo que se leía como un corte. Ahora se encaja DENTRO de la caja conservando su relación: 156x136, con 22 px de césped a los lados.

Los adornos del césped se sembraban solo dentro del mundo. $dx = 8 + \text{azar} * (W - 16)$, o sea entre 0 y W. Pero el césped ahora llega hasta el bosque, así que la franja entre la cerca y los árboles quedaba **pelada**, con un rectángulo perfecto marcando dónde se acaban las matitas y las flores. Ahora se siembran con 3 celdas de desborde y la cantidad sube en proporción al área (110 – 223) para que la densidad no baje. Van a profundidad -999,5, debajo del anillo, así que los que caen bajo el bosque no se ven ni estorban.

La vista previa no dibujaba los adornos, así que no podía delatar este fallo — el “corte” era invisible para la herramienta que existe justamente para verlo antes de deployar. Ahora los dibuja, con el mismo azar y las mismas cuentas que el juego.

Nota sobre el diagnóstico de este día

Ante “¿qué ves mal?” respondí analizando la INTERFAZ de una captura que resultó ser **un recorte**: di por rotos unos cortes de HUD que eran del recorte, no del juego. Dirección se refería al mundo. Antes de diagnosticar sobre una imagen hay que preguntar si es la ventana entera. (Lo que sí quedó verificado de aquello, y sigue en pie como deuda: en todo index.html hay **una sola** @media, y es para un modal — el HUD no tiene ninguna regla de adaptación al ancho.)

La diferencia de color era el fondo de la cámara, y lo había puesto yo

Dirección: “*hay diferencia en el color*”. Medido:

césped (media de grass_a) `rgb(50, 128, 50)`
fondo de la cámara `rgb(47, 90, 40)` ← 38 niveles más oscuro

Yo había elegido ese fondo muestreando **la copa de los árboles** y oscureciéndola un 18%, pensando en “bosque lejano”. Pero el fondo no se ve donde hay bosque: se ve por los **huecos**. Y un hueco con 38 niveles de diferencia contra el césped canta como una mancha apagada.

Peor: el mosaico del bosque lo dibujaba sobre **lienzo transparente**, y tiene un 14% de claros por el raleo. Cada claro dejaba ver ese verde oscuro, así que todo el bosque de más allá del mapa salía moteado. Dentro del anillo esos mismos claros dejan ver el césped de verdad — o sea que el mosaico no imitaba lo que imita.

Dos correcciones: el fondo de la cámara pasa a ser **el color medio del césped** (0x328032), y el mosaico se dibuja **sobre baldosas de césped** en vez de sobre transparente. Así un hueco se lee como suelo en los dos sitios.

Regla que sale de acá: **un color de relleno no se elige por lo que representa, sino por lo que va a aparecer al lado**. El fondo estaba junto al césped, no junto a las copas.

La laguna cortada estaba en el ARTE, no en el código

Dirección insistió tres veces con “*laguna cortada*”, y tenía razón las tres. Yo busqué la causa en el código —proporción, atlas, profundidades, recortes— y no estaba ahí.

pond.png venía con la forma **tocando el borde izquierdo del lienzo**: 12 filas de orilla seccionadas en plano, como si el PNG se hubiera recortado un poco corto. Ningún cambio de código podía arreglarlo porque el corte estaba dentro del sprite. Se ve de un vistazo ampliándolo x5 sobre césped, que es lo que había que hacer desde el principio en vez de teorizar.

tools/arreglar-laguna.py lo cierra sin inventar dibujo: ensancha el lienzo y **prolonga la curva real del contorno**. Toma las filas sanas de arriba y abajo del corte, ajusta por mínimos cuadrados la parábola que describe la orilla ($x = 0,0247 \cdot (y - 40,5)^2 - 0,26$) y la continúa por dentro del corte. Guarda pond_original.png como copia.

La primera versión metía una media elipse y salía **un nudo**: la elipse moría de golpe contra un contorno que ya venía retrocediendo. Prolongar la curva que ya existe, en vez de superponerle una forma inventada, es lo que hace que empalme sin escalón.

Atlas reconstruido (el sprite pasó de 107 a 108 px) y $v=47$.

Lección del día, la más cara. Ante un fallo visual, mi reflejo fue buscarlo en el código: probé proporción del sprite, integridad del atlas, orden de profundidades y recortes de cámara. El fallo estaba en el PNG y se veía **abriendo el PNG**. Cuando algo “se ve cortado”, lo primero es mirar el arte a tamaño grande; el código es la segunda hipótesis, no la primera.

La laguna cortada: cuatro hipótesis descartadas y un diagnóstico en pantalla

Dirección: “*de verdad ves la laguna entera?*”. No. Y peor: **había dicho que sí sin mirarla**, que es exactamente el error por el que venía pidiendo disculpas todo el día. Ampliada x4, sigue cortada.

Lo medido sobre su captura, que es lo único firme:

- El agua empieza en **x=313 exacto en el 94% de las filas**: una recta perfecta.
- El agua ocupa **126 px de ancho por 147 de alto**. El sprite es 108x93, o sea **siempre más ancho que alto**. Que salga más alto que ancho solo puede significar que le falta un trozo por el costado, y la cuenta da **exactamente una celda (42 px de mundo)**.

Hipótesis descartadas **con datos**, no por intuición:

1. *La proporción del sprite* — corregida, y no era.
2. *El arte* — el sprite tenía un corte real de 16% de su alto, se cerró prolongando la curva del contorno... y el corte del juego sigue siendo del 94%. No era (o no era solo eso).
3. *El atlas* — frames correctos, cero solapes, y el atlas **del servidor** ya sirve pond con 108 px: el deploy llegó.
4. *El código* — pondImg solo se crea y se reposiciona; no hay setCrop ni máscara sobre ella.

Como llevo cuatro teorías erradas mirando imágenes, se acabó teorizar: se añade un diagnóstico que se abre con **?laguna=1** en la URL y dibuja el recuadro donde el juego cree que está la laguna (rojo), sus celdas (azul) y la grilla (amarillo), más su tamaño de textura y de dibujo. Con eso se distingue de una vez entre “se dibuja entera y algo la tapa” y “se dibuja ya recortada”. No se enciende solo, y no necesita consola.

La isla no debía estar en el atlas (y puede ser la causa de la laguna)

Buscando “cero pedidos sueltos” metí isla dentro del atlas. boot.js ya lo desaconsejaba en un comentario que no leí: *“COSTA de la isla: imagen grande y aparte, NO va al atlas (mide 1190x854)”*.

Consecuencias: ella sola empujaba el lienzo del atlas a **4096x1934 = 7,9 millones de píxeles** (unos 31 MB de textura RGBA en memoria) y reordenaba todo el empaquetado. Fuera del atlas queda en **4096x1142** y 764 KB. Encima, con el bosque puesto la isla ni se dibuja: solo la usa el modo mar.

Por qué puede ser la causa del corte de la laguna: si el navegador reescala una textura tan grande, el drawImage(src, fr.x, fr.y, ...) que desempaqueta cada frame lee de **coordenadas equivocadas**, y cada sprite sale con un trozo de otro sitio. Encaja con lo observado: el recuadro del sprite tiene el tamaño correcto (158x136 confirmado por el diagnóstico) pero su contenido está desplazado y recortado. No está demostrado, pero es la primera hipótesis que explica el síntoma exacto.

El corte de la laguna era el BOSQUE — lo encontré dirección, no yo

Después de cinco hipótesis más erradas (proporción del sprite, el arte, el atlas, las profundidades, la caché), dirección propuso la sexta: *“no será por los cambios que hicimos en el bosque”*. Se probó con el interruptor que ya existía, ?sinbosque=1, y **la laguna sale entera**.

La causa es memoria de textura. Con el bosque puesto la escena creaba:

textura

renderTexture del césped

renderTexture del bosque

atlas desempaquetado

total

Cuando eso se agota, las subidas de textura fallan en silencio y los sprites salen a medias: la laguna se dibujaba con el tamaño correcto (el diagnóstico confirmó 158x136) pero con su contenido recortado. Encaja con todo lo observado, incluido que mejorara al reducir el atlas.

Dos recortes, sin perder nada visible:

- **El césped no necesita cubrir el mapa.** Solo se ve la franja entre la cerca y el primer árbol; de ahí para afuera lo tapa el bosque, y donde el bosque tiene claros el fondo ya es del color del césped. De 1638x1680 a **1050x840**: de 10,5 a 3,4 MB.
- **El anillo dibujado se recorta a 7 celdas** (BOSQUE_RT_CELDAS). Más allá, el mosaico repite el mismo patrón con las mismas leyes y se ve idéntico, pero no cuesta memoria porque se repite. De 1600x1600 a **1302x1092**: de 9,8 a 5,4 MB.

11,5 MB menos, de 39 a 27.

Lección, y es la más importante del día. Llevaba cinco hipótesis mirando el síntoma (la laguna) y ninguna miró el CONTEXTO (qué cambió al mismo tiempo). Dirección lo resolvió con una pregunta de correlación: *¿no será por lo que acabamos de tocar?*. Y la prueba costó diez segundos porque el interruptor ya existía. **Ante un fallo nuevo, lo primero es apagar lo último que se añadió.**

El día y la noche solo pasaban dentro del corral

Dirección: *“dentro del corral es de un día y fuera del corral es otro horario, y se ve claro”*. Exacto, y el motivo estaba a la vista en una línea:

```
this.cielo = this.add.rectangle(0, 0, GF.WORLD_W, GF.WORLD_H, ...)
```

El velo de día/noche medía **solo el mundo jugable**, 714x504. Cubría el interior de la cerca y nada más: el bosque, la franja de césped y el mosaico se quedaban en mediodía perpetuo mientras la granja anochecía. Nunca se había notado porque hasta ayer fuera de la cerca solo había mar.

Ahora el velo mide 9000x9000 y va centrado en el mismo punto que el mosaico: **la hora es una sola en toda la pantalla**, la cámara vaya donde vaya.

Flores también fuera de la cerca

El desborde de adornos sube de 3 a 4 celdas —lo mismo que ahora cubre el césped— y la cantidad se calcula por ÁREA (270 en total). Así la franja de fuera queda **igual de poblada** que la de dentro, y la cerca deja de marcar dónde se acaban las florcitas.

Patrón que se repite en los tres fallos de hoy —el pasto, los adornos y ahora el velo—: **todos estaban dimensionados al mundo jugable**, porque hasta ayer no había nada más allá de la cerca. Al meter el bosque, todo lo que medía WORLD_W x WORLD_H se quedó corto a la vez. Si aparece otro fallo de este estilo, ahí es donde hay que mirar primero.

Las flores de fuera existían, pero el bosque se las comía

Dirección: *“falta ver si se puede poner por encima del césped, fuera del corral, las mismas flores que hay dentro”*. Estaban puestas —el reparto ya llegaba a 4 celdas más allá de la cerca— pero no se veían, y el motivo es de CAPAS, no de posición:

- los adornos van a profundidad **-999,5**
- el bosque se dibuja a **-999**, o sea **por encima**

Los que caían bajo los árboles no existían para el jugador. Y como el reparto era uniforme sobre todo el rectángulo, la mayoría de los de fuera caían justo ahí: solo el césped visible mide 2,3 celdas más allá de la cerca, el resto es bosque.

La solución no es subirlos de capa —quedarían flotando encima de los troncos— sino **sembrarlos solo donde se ven**. Se añade dentroDelClaro(x, y), que usa la misma métrica que dibujarBosque() y vive en un solo sitio para que no se puedan desincronizar. Los adornos ahora:

1. descartan cualquier posición que quede bajo el bosque,
2. estiman el área ÚTIL muestreando, y calculan la cantidad sobre ESA área.

Resultado: 173 adornos, **1 cada 3.263 px² dentro y fuera** — la misma densidad, y ninguno tirado.

Tercer fallo seguido con la misma raíz: algo pensado para un mundo que se acababa en la cerca. Van el pasto, el velo de día/noche y ahora los adornos.

De día mariposas, de noche luciérnagas

Dirección: *“cuando hay claridad, las mariposas revoloteando; cuando sea de noche, reemplazadas por luciérnagas con cierto halo de luz alrededor, el mismo comportamiento. Y que no desaparezcan bruscamente, sino con un fade out”*.

Decisión de implementación: **no son dos bichos, es uno con dos trajes**. Cada mariposa lleva ahora un segundo dibujo —la luciérnaga— en la misma posición, y lo único que cambia con la hora es cuál de los dos se ve. Así se conserva TODO el comportamiento que ya estaba: el merodeo, el posarse en las flores, y la función de guía que señala lo accionable. Si fueran entidades separadas habría que duplicar esa lógica y se desincronizarían.

- **El halo** son cuatro círculos concéntricos de alfa decreciente (0,055 → 0,30) más un cuerpo casi blanco. Graphics no hace degradados, pero apilados de fuera hacia dentro dan el resplandor.
- **Late, no brilla parejo**: la intensidad oscila con un seno, y cada luciérnaga lleva su propia fase para que no parpadeen todas a la vez.
- **El relevo es un fundido de 6 segundos** (nocheMezcla, de 0 a 1), enganchado al **mismo umbral que enciende los faroles**: todo lo nocturno pasa junto y no hay dos momentos distintos.
- Si entrás con el juego ya de noche, se aplica seco — el fundido es para la transición, no para la carga.

FX_LUCIERNAGAS = 1 en config; a 0 las mariposas vuelan también de noche.

17/8 — La granja es CUADRADA: 15×15, y el anillo de 16 expansiones

Dirección: “*el ancho de la granja, incluyendo el corral, debe ser el mismo de largo que de ancho, y el rectángulo que forma el bosque también, y el conjunto entero también, que es 1600*”.

La granja era 17×12. No cumplía ninguna de las tres condiciones.

Por qué 15×15 y no 12×12 ni 16×16

Los dos números que propuso dirección no entran, y por la misma razón: **el contenido mide 15 celdas de ancho**, así que el interior necesita 15 columnas y el mundo 17 como mínimo... salvo que se re-maquete. Se re-maquetó, y entonces la restricción que manda es otra: **el lado tiene que partirse en 3 partes iguales**, porque el anillo de expansiones lleva 3 bloques por lado.

granja

12×12

15×15

16×16

17×17

18×18

15 y 18 son los únicos que se parten en 3. **15 gana porque la granja casi se triplica** (225 → 625) contra el +78% de 18×18, y porque 18×18 devolvía 6 filas vacías al nivel 1 — justo lo que se sacó ayer al comprimir el mundo.

El anillo: 16 bloques IDÉNTICOS de 5×5

Dirección: “*un lateral, sin contar las esquinas, sea tres expansiones... las cuatro esquinas son cuatro... entre todo serían dieciséis*”, y después “*cada expansión medirá cinco por cinco*”.

Ese segundo dato corrige la propuesta anterior, que tenía la banda en 4 y dejaba los bloques desaparejos (laterales 5×4, esquinas 4×4) sin ninguna necesidad. Con la banda en 5 sale la figura que se explica sola:

El mapa entero es una grilla de 5×5 BLOQUES de 5×5 CELDAS. El corral es el 3×3 del centro; las 16 expansiones son el marco.

Los 16 bloques valen exactamente lo mismo (25 celdas), así que no hay que explicarle a nadie por qué una esquina rinde menos que un lateral. **Por qué no entra una expansión 17:** el bosque quedaría en 4,25 celdas de árboles puros descontando el anillo de césped, y por debajo de eso deja de leer como bosque.

GF.EXPANSIONES (los 16 bloques), GF.mundoCon(compradas) y GF.celdaComprada(col,row,compradas) quedan escritos en config. **Todavía no los lee el juego** — esta entrega es solo la granja cuadrada.

El origen se corre. Al comprar por la izquierda o por arriba el claro pasa a tener columnas y filas NEGATIVAS. Las coordenadas guardadas de los objetos no se tocan: se mueve el BORDE, no el contenido, así que ningún layout guardado se rompe.

El re-maquetado interior

El interior pasó de 15×9 (tumbado) a 13×12 (casi cuadrado): sobran 3 filas y faltan 2 columnas.

- **Las parcelas giran:** de 4×3 a 3×4. Son las mismas 12, pero libera la columna que el rincón del correo necesitaba, porque los árboles ahora empiezan en la columna 10 en vez de la 11.
- **Todo el bloque natural baja:** árboles a las columnas 10-13 filas 2/4/6 (el mismo ritmo de antes), cantera a las columnas 10 y 12 filas 8/10/12, y las vetas a las columnas 11 y 13 filas 9/11/13, en orden de tier hacia abajo.
- **La laguna baja** a las filas 10-12 y el sitio de pesca la acompaña.
- **El establo baja a la fila 9.** Con la grilla de parcelas de pie su tejado las pisaba: mide 2,5 celdas de alto y las tapaba dos filas más arriba de su base. Lo delató checkLayout, no el ojo.

Verificado antes de subir

- checkLayout pasa de **6 avisos a 1**, y el que queda (“*barn se sale de la cerca*”) **ya estaba en git HEAD**: es el tejado del granero asomando sobre la cerca de arriba, que es como se ve desde siempre. Comprobado stasheando los cambios y volviendo a correrlo.
 - Los seis archivos del juego se **ejecutan** con Phaser stubbeado, no solo node --check — que ya nos dejó pasar un bloque huérfano entero. ui.js falla por falta de DOM real, y falla igual en git HEAD.
 - **Memoria de textura: 8,79 → 9,22 MB (+0,43).** El bug de la laguna cortada fue por llegar a 39 MB y que las texturas se subieran a medias, así que ahora se mide en cada cambio de tamaño. **Aviso para cuando lleguen las expansiones: al final (25×25) esto da 17,6 MB.** Sigue lejos de 39, pero es el doble de hoy y hay que volver a medirlo entonces.
-

17/8 — AUDITORÍA: la vara de los minerales no sigue el ancla

Dirección: “*siempre todo tiene que encajar con el ancla, la fórmula. Nada está armado en torno a otra cosa. Tiene todo que ser parte de un puzzle, de la columna vertebral.*”

Antes de montar la escalera diaria/semanal/mensual del tablón fui a verificar el ancla sobre el código, porque el tablón valora cada pedido con priceOf() — o sea, con PRICE. Si la vara está torcida, todo lo que cuelgue de ella sale torcido, y no se nota nunca **porque los materiales no se venden**: no hay mercado que lo delate.

state.js ya declara la fórmula: **valor = horas del reloj × 20 + costo de la herramienta.**

material	reloj
madera	1,5 h
piedra	2 h
bronce	8 h
hierro	12 h
oro	14 h
diamante	18 h
netherita	24 h

Madera y piedra están clavadas en la fórmula. Los cinco minerales no.

Lo que eso significa en la mesa

Los picos tienen dur: 1 — **un pico por picada** — y cada pico se craftea con el mineral del tier anterior, así que el costo se compone hacia arriba. Con los precios del propio juego:

mineral
bronce
hierro
oro
diamante
netherita

Picar cualquier mineral por encima de la piedra da pérdida. No es un desajuste de tabla: es que toda la escalera de minería está por debajo del ancla, y el jugador que sube por ella se empobrece respecto del que se queda talando.

El arreglo que propongo

El ancla dice cuánto puede costar la herramienta por uso: $PRICE - 20 \times \text{horas}$. O sea 50, 60, 190, 630 y 760. Los picos cuestan hoy entre 2,5 y 6 veces eso.

Un solo número de diseño: todos los picos duran 5 picadas, y la receta se deriva para que el costo por uso sea exactamente el que el ancla permite (5×50 , 5×60 , 5×190 ...).

Ventajas: no se toca ningún precio, así que nada de lo que lee `priceOf()` se mueve; la escalera entera queda sobre la fórmula de una sola vez; y el desgaste pasa a ser un sumidero de material recurrente, que es justo lo que faltaba.

Lo que NO hay que hacer es dejar dur: 1 y subir los precios a la columna “fórmula”: el diamante pasaría a valer 5.877 y cualquier pedido del tablón que lo pida se vuelve absurdo.

Auditoría reproducible: `node tools/auditar-precio-sombra.js`. Resuelve el costo de la herramienta por iteración, porque es recursivo (el pico de oro se hace con bronce, que vale según esta misma tabla).

APLICADO — y el campo que había que arreglar primero

Al ir a subir la durabilidad apareció el verdadero fallo: **dur en PICK_DEF estaba muerto**. El campo existía desde siempre, pero craftPick hacía `G.picks.dur[id] = pickCount(id) + 1` — un 1 fijo — y nadie leía `pd.dur`. O sea que poner `dur: 5` no habría hecho absolutamente nada, y el arreglo del ancla habría sido un **no-op silencioso** anunciado como resuelto.

Arreglado en una línea (`+ (pd.dur || 1)`), y el toast ahora dice “+5” en vez de “+1”.

Recetas derivadas (presupuesto = lo que el ancla permite por uso × 5 usos), pagadas **solo en material** para que el desgaste sea además sumidero:

pico

Bronce

Hierro

Oro

Diamante

Netherita

Los cinco minerales pasan de dar pérdida a rendir **exactamente 20 plata/hora**, igual que una parcela y que un árbol. El de netherita salía 3,84 diamantes: redondear a 4 lo dejaba un 7% por debajo, así que se parte en 3 diamantes + 18 piedra y da el presupuesto clavado.

El pico de piedra **no se toca**: ya estaba sobre el ancla ($46 - 40 = 6 =$ su costo) y el tutorial consume 11, así que moverlo cambiaba el arranque sin necesidad. Verificado: `sim-tutorial.js` sigue cerrando en 18,1 h.